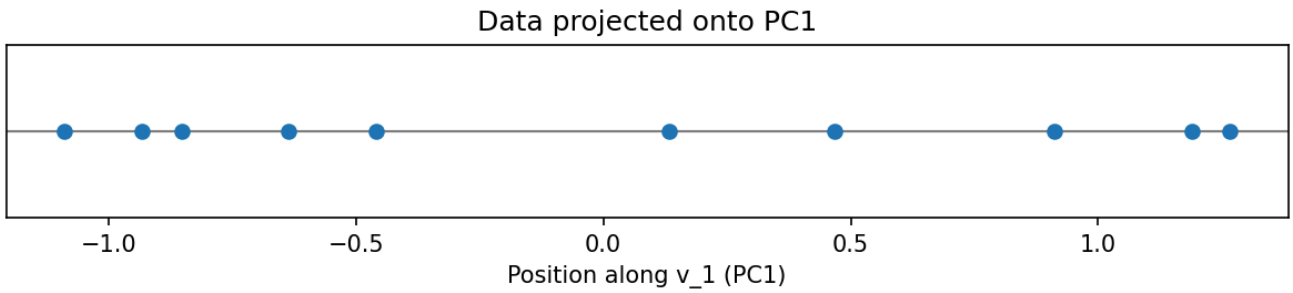
Assignment 3: Dimensionality Reduction / PCA

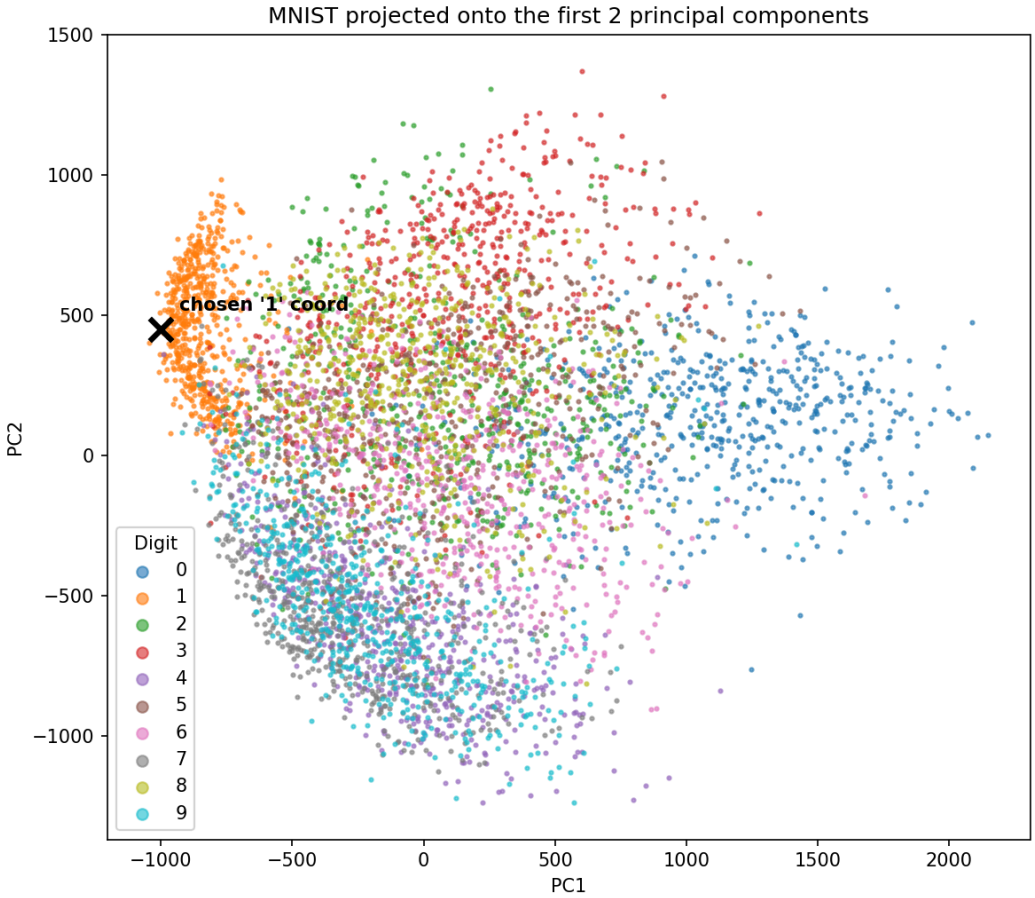
Evan Fitzhugh · BME 205

Part 1: PCA and eigendecomposition

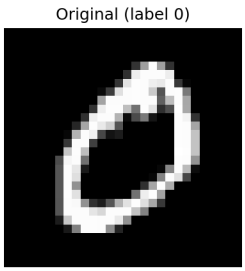


`projection_1D.png`: the centered data projected onto v_1, plotted on a number line.

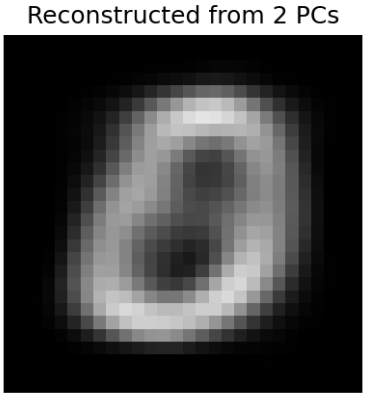
Part 2: PCA on MNIST



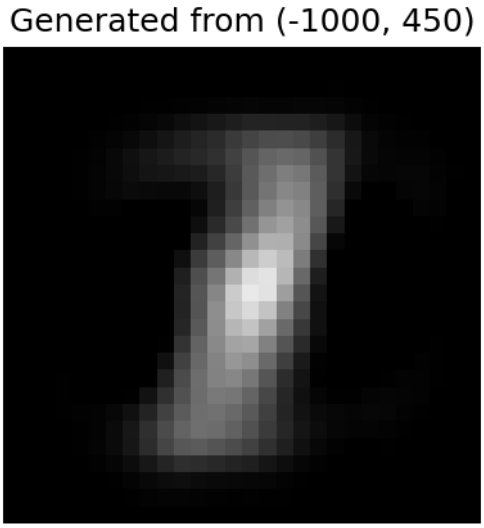
`MNIST_PCA_2D.png`: the 6000 MNIST images projected onto the first 2 PCs, colored by digit label.



MNIST_original.png
Original image 0



MNIST_reconstructed_2PC.png
Image 0 reconstructed from 2 PCs



MNIST_reconstructed_1_from_coord.png
Image generated from the point (-1000, 450)

How I know that my results are correct

Method

To make sure that my results are correct, I calculated each value again with a different method. When two different methods give the same value, the risk of an error in one method is small. NMF does not give one correct result. Thus, I also calculated NMF again from different start points.

Part 1: PCA and eigendecomposition

I calculated PCA with my steps. I centered the data, calculated the covariance matrix, and found its eigenvectors with NumPy. The eigenvalues (0.8308 and 0.0027) are the same as the scikit-learn explained variance values. The variance ratios (0.9968 and 0.0032) and the 10 projected values are also the same as the scikit-learn values.

The eigenvector v_1 is the same as PC1. The eigenvector v_2 has the opposite sign from PC2, but it points along the same line. If v is an eigenvector, then -v is also an eigenvector with the same eigenvalue. Scikit-learn changes the sign of a component when the entry with the largest absolute value is less than 0, whereas NumPy does not.

But whaever the case, the projection plot keeps the sequence of the points along the line in the data plot, so it is considered right.

Part 2: PCA on MNIST

I calculated the eigenvectors of the covariance matrix of the 6000 images with NumPy. The two largest eigenvectors are the same as PC1 and PC2 (absolute dot product = 1.0). The two largest eigenvalues (339,839 and 245,995) agree with the scikit-learn explained variance to more than 6 digits.

I also calculated the reconstruction of image 0 with my equation (mean + coordinates × components). The result is the same as the inverse_transform result. The reconstructed image shows a 0, but the image is not clear. The cause is that 2 PCs keep only approximately 17% of the variance.

In the 2-D plot, the 1s make a clear group on the left side. I selected the point (-1000, 450) at the left edge of this group. The image from this point shows a very rough 1. But, its enough to say the PCA axes contain information about the shape of each digit.

Part 3: NMF on the dog data

With the default limit of 200 iterations, NMF gives a convergence warning. With max_iter=1000, NMF stopped after 791 iterations without a warning. Each row of the normalized W has a sum of 1. In the table, the row sums are between 0.99 and 1.02 because I rounded each value to 2 decimals.

NMF can give different results from different start points. I calculated NMF again from 5 random start points. All 5 results had the same reconstruction error (663.4) as my first result. But the clade values changed by a maximum of 0.26. Thus, NMF has more than one equally good solution, and my table is one of these solutions. With random_state=0, the script gives the same table each time.

The primary result did not change. In all 6 results, Wolf, Golden Jackal, Asian Spitz, and Basenji have most of their ancestry in the same component (0.52 to 0.77). Spaniel has only 0.03 to 0.07 of this component. This result agrees with dog genetics. The wild canids and the basal breeds have DNA that is different from most European breeds (at least according to some light googling).

Acknowledgement

I used Claude (Anthropic) to help write and debug my scripts, and to compile the images and this explanation into this PDF.